

实验：开发环境篇

涵盖 1.1 / 1.2 / 1.3 全部实验任务 · 必做实验

实验 1.1 平台认知与 ruyi 环境安装

实验目标

- 确认 host 与 target 的架构和系统信息
- 查询交叉编译器目标三元组与 sysroot 信息
- 完成 host / target / sysroot 三要素对照表
- 安装 ruyi，更新软件源，查询并验证工具链

实验环境

项目	要求
主机环境	Linux x86_64（推荐），已安装 ruyi
目标板	LicheePi 4A
硬件连接	网络可达、USB 数据线、USB-TTL 串口模块
软件依赖	ssh、file、curl

任务 1：采集 host 与 target 信息

在主机执行：

```
$ uname -m
$ hostname
```

在板端执行：

```
$ uname -m
$ hostname
$ head -n 5 /etc/os-release
```

任务 2：安装 ruyi 并查询工具链

```
$ command -v ruyi
$ ruyi --version
$ ruyi update
$ ruyi list | grep toolchain
```

任务 3：查询工具链与 sysroot

```
$ riscv64-unknown-linux-gnu-gcc -dumpmachine
$ riscv64-unknown-linux-gnu-gcc --version | head -n 1
$ riscv64-unknown-linux-gnu-gcc -print-sysroot
```

若 `-print-sysroot` 为空：

```
$ riscv64-unknown-linux-gnu-gcc -v -E - < /dev/null 2>&1 | head -n 30
```

任务 4：最小编译验证

```
int main(void) { return 0; }
```

编译后使用 `file` 检查输出文件架构，应显示 RISC-V ELF。

任务 5：填写对照表并回答问题

完成 host / target / sysroot 对照表（项目 / host / target / sysroot 四列），并回答课堂练习中的两句话判断题。

验收标准

验证项	预期现象
host 架构	与板端不同（通常 x86_64）
target 架构	riscv64
三元组	含 linux-gnu
sysroot	有路径或可查证的查找方式
对照表	完整无空项

实验 1.2 镜像烧录、首次启动与开发通道

实验目标

- 校验镜像并执行安全烧录
- 完成首次启动后的系统、网络、时间检查
- 建立串口连接作为备用通道
- 配置 SSH 密钥登录并完成 SCP 双向传输

实验环境

项目	要求
镜像	与 LicheePi 4A 匹配的课程指定 RevyOS 镜像
烧录工具	与镜像说明匹配
串口	3.3V USB-TTL 模块 + 终端程序
SSH	主机具备 <code>ssh</code> 、 <code>scp</code>

任务 1：校验并烧录镜像

```
$ sha256sum <image-file>
# 与发布方校验值比对，不一致则停止
```

按该镜像版本发布说明执行烧录。一次只连接一块待烧录板。保存完整日志直到工具明确报告成功。

任务 2：首次启动与系统检查

```
$ cat /etc/os-release      # 确认 RevyOS
$ uname -a                 # riscv64
$ id                       # 记录当前用户
```

```
$ hostnamectl          # 主机名
$ df -h               # 存储空间
```

任务 3：检查网络和时间

```
$ ip -br addr
$ ip route
$ timedatectl
```

任务 4：apt 软件源检查

```
$ grep -R --no-filename -h '^deb ' /etc/apt/sources.list /etc/apt/sources.list.d 2>/dev/n
$ sudo apt update
```

先确认链路、地址、路由和时间正常，再执行更新。失败时按 DNS → 时间 → 网络 → 仓库依次排查。

任务 5：建立串口连接

断电接线：GND-GND、TX → RX、RX → TX。打开终端工具，按板卡资料设置波特率。

任务 6：配置 SSH 密钥登录

```
# 板端启用 SSH
$ sudo systemctl enable --now ssh

# 主机生成密钥对
$ ssh-keygen -t ed25519 -C "riscv-course"

# 首次连接——核对指纹后确认
$ ssh user@board-ip

# 部署公钥
$ ssh-copy-id user@board-ip
```

任务 7：SCP 双向传输

```
# 上传并校验
$ printf 'transfer test\n' > test.txt
$ sha256sum test.txt
$ scp test.txt user@board-ip:/tmp/
$ ssh user@board-ip 'sha256sum /tmp/test.txt'

# 从板端下载
$ scp user@board-ip:/etc/os-release ./board-os-release.txt
```

验收标准

验证项	预期现象
镜像校验	与发布值一致
烧录结果	工具报告成功
<code>uname -m</code>	<code>riscv64</code>
网络状态	有有效 IP 和默认路由
SSH 密钥登录	免密登录成功
SCP 双向	两端哈希一致

实验 1.3 ruyi device provision 适用边界

实验目标

- 查询当前 `ruyi` 版本的 device provision 支持状态
- 完成六项 risk 检查并做出有依据的决策
- 对比手工烧录流程与自动化 provision 的适用场景

本实验的重要特点：不一定需要执行自动写入。「当前版本不支持该设备」并描述手工替代方案同样是合格的实验结论。

任务 1：读取当前版本帮助

```
$ ruyi device --help
$ ruyi device provision --help
```

若当前版本没有这些子命令，记录版本与帮助输出——这本身就是「当前版本不具备该路径」的有效结论。

任务 2：查询设备与变体

使用帮助中提供的列表或查询命令，记录设备标识、变体、支持的镜像和 provision 方式。板卡商品名相同但硬件版本或配置不同，可能需要不同变体。

任务 3：执行六项风险检查

1. 设备是否在当前软件源中明确列出？
2. 变体是否与实物一致？
3. 写入目标是否唯一且可确认？
4. 重要数据是否已备份？
5. 供电和数据连接是否稳定？
6. 是否已准备串口或其他恢复通道？

六项全部确认后，方可执行自动 provision。任一项存疑都应停止自动写入，改用 1.2 手工流程。课程接受任何有依据的决策路径。

任务 4：场景决策

分别对以下场景写出决策理由：

- 1. 设备完全匹配且有重要数据 → 手工流程
- 2. 设备未列出但名称相似 → 手工流程，记录不支持原因
- 3. 设备列出且为新空白存储 → 可考虑 provision

验收标准

验证项	预期现象
帮助检查	确认当前版本能力
设备查询	明确支持或不支持的结论
风险清单	六项均有答案
路径选择	决策理由清楚，与手工流程对比明确